

COLLEGECONNECT VALIDATION REPORT

This document provides a step-by-step walkthrough of the CollegeConnect multi-role application, including role-based workflows, system validations, and database checks.

1. CORE WORKFLOW WALKTHROUGH

1.1 User Registration & Validation

- * The system supports 5 user roles: student, coordinator, faculty_admin, super_admin, and guest.
- * Student/Guest: Accounts are verified by default (isVerified: true).
- * Coordinator/Faculty: Accounts require admin verification before accessing publishing tools.
- * ID Proof Verification (OCR): During registration, the backend runs Tesseract OCR on the uploaded Student College ID. If the name or college details extracted do not match the form input, registration is rejected with: "Student College ID and details are not matched"
- * Registration Limits: A maximum of 2 Faculty Admins and 4 Coordinators are allowed per college.

1.2 User Login & Security

- * Login calls the POST /api/auth/login endpoint.
- * Passwords are encrypted using bcryptjs (salt rounds: 10).
- * A secure JWT token is generated on success (expires in 24 hours).
- * The Flutter app inspects the role in the JWT payload and routes the user:
 - Coordinators: Routed to Coordinator Dashboard (Event publishing, scanner, pending payments).
 - Faculty: Routed to Approval Dashboard (Pending events, pending coordinators)
 - Students: Routed to Event Browsing, tickets, and attendance history.

1.3 Event Publishing & Rejection Workflow

1. Creation: A coordinator creates an event. If it's a paid event, they input the fee and their UPI ID. The event is saved as isApproved: false.
2. Alert: An email notification is sent to the Faculty Admin.
3. Rejection: If the Faculty Admin rejects the event or its proposed updates, the backend sets rejectionReason: "<Feedback>" and sends a rejection email to the coordinators of the college.
4. Edit / Resubmit:
 - On the event details page, the coordinator sees a red "REJECTED / ACTION REQUIRED" warning banner showing the feedback.
 - Clicking "Edit" on the banner enters edit mode. During this flow, the bottom button changes to "Resubmit". Saving changes resets the status to pending.
5. Dismiss & Keep Original (For already approved events):
 - If the event was already published and subsequent edits were rejected, the coordinator can click "Dismiss & Keep Original".

- This dismisses the warning banner, discards the edits, and reverts the view back to the original approved event details.

1.4 Student Registrations & Payments

- * Paid Events: When registering, the student enters a UPI transaction reference ID. The registration status starts as pending.
- * Payment Approval: The coordinator sees the transaction ID under the "Pending Payments" tab. Upon verifying the payment, they click "Approve". The status updates to completed, and a confirmation email is sent.

1.5 QR Scanner & Attendance

- * Ticket Generation: Upon successful registration, the app generates a ticket with a unique QR code (cc_qr_<uuid>).
- * Attendance Scanning: The coordinator opens the QR Scanner in the app. Scanning the student's ticket triggers:
POST /api/events/:eventId/attendance/:attendanceId/verify
This registers the timestamp and marks the student's attendance as verified.

2. LIVE VALIDATION TEST LOG (E2E Integration Test)

A programmatic end-to-end integration test was executed on the local server to validate all API endpoints. All tests completed successfully.

- [✓] Step 1 : Register Student (isVerified defaults to true) -> PASSED
- [✓] Step 2 : Register Coordinator (requires verification) -> PASSED
- [✓] Step 3 : Register Faculty (requires verification) -> PASSED
- [✓] Step 4 : Super Admin Logs In (returns JWT token) -> PASSED
- [✓] Step 5 : Super Admin approves Faculty (isVerified -> true) -> PASSED
- [✓] Step 6 : Faculty Admin approves Coordinator (isVerified -> true) -> PASSED
- [✓] Step 7 : Coordinator Publishes Event (status is Approved = false) -> PASSED
- [✓] Step 8 : Faculty Admin approves Event (status is Approved = true) -> PASSED
- [✓] Step 9 : Coordinator edits Event (changes stored in pendingUpdates) -> PASSED
- [✓] Step 10: Student registers for Paid Event (payment status is pending) -> PASSED
- [✓] Step 11: Coordinator approves Payment (payment completed, email sent) -> PASSED
- [✓] Step 12: Coordinator scans student ticket (attendance marked) -> PASSED
- [✓] Step 13: Coordinator dismisses edits (reverts to original) -> PASSED

3. SMTP EMAIL VERIFICATION STATUS

Email notifications are configured to use Gmail SMTP over SMTPS on Port 465.

- * Transporter Connection Mode: Secure TLS/SSL handshake (secure: true).
- * Environment Variable Cleaning: Automatically trims double/single quotes and spaces from the environment settings to support direct copying into host configurations.
- * Test Verification:

Target Email: nagawellyrohit@gmail.com
Calling sendEmailNotification...
[Email Sent] Message ID: <92ea012f-a9c6-8ae2-7fc1-237a09ea7074@gmail.com>
Email send result: { success: true, messageId: '<92ea012f-a9c6-8ae2-7fc1-237a09ea7074@gmail.com>' }

----- 4. LAYOUT VERIFICATION (iPhone Screen size) -----

The buttons inside the rejection box have been updated from a horizontal Row to a responsive Wrap layout:
* Horizontal Spacing: 8.0
* Run Spacing: 8.0
* Alignment: WrapAlignment.end
* This guarantees that when both the "Edit" and "Dismiss & Keep Original" buttons are displayed, they will stack vertically on small devices without causing any RenderFlex layout overflow assertions.

=====
END OF REPORT
=====